```
#include <iostream>
#include <iomanip>
#include <string>
#include <limits>
#include <stdexcept>
using namespace std;

constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;

enum class Zone {
   Inner = 1,
   Middle = 2,
   Outer = 3
};

// Check peak hours
bool isPeakHour(int hour, int minute) {
   if (hour < 0 || hour > 23 ||
      minute < 0 || minute > 59) {
      throw invalid_argument("Invalid time");
   }

   int time = hour * 60 + minute;

   return (time >= 480 && time < 600) ||
         (time >= 1020 && time < 1200);
}
```

```
// Calculate fare in integer paise
long long calculateFare(int zoneCode, int hour, int minute) {
    long long fare;

    switch (zoneCode) {
        case 1:
            fare = 1000;  // Rs. 10
            break;
        case 2:
            fare = 2000;  // Rs. 20
            break;
        case 3:
            fare = 3000;  // Rs. 30
            break;
        default:
            throw invalid_argument("Unknown zone");
    }

    if (isPeakHour(hour, minute)) {
        fare = fare * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    }

    return fare;
}

// Check minimum balance
bool canTravel(long long balance, long long fare) {
```

```
    return fare >= 0 &&
           balance >= fare &&
           balance - fare >= MIN_BALANCE_PAISE;
}

// Recharge validation
bool recharge(long long& balance, long long amount) {
    if (amount <= 0 ||
        balance > numeric_limits<long long>::max() - amount) {
        return false;
    }

    balance += amount;
    return true;
}

// Validate numeric input
bool parseHour(const string& input, int& hour) {
    try {
        size_t position;
        int value = stoi(input, &position);

        if (position != input.size() ||
            value < 0 || value > 23) {
            return false;
        }

        hour = value;
```

```
        return true;
    }
    catch (...) {
        return false;
    }
}

void printMoney(long long paise) {
    cout << "Rs. " << paise / 100 << "."
         << setw(2) << setfill('0') << paise % 100
         << setfill(' ');
}

int main() {
    cout << "NAMMA METRO FARE LOGIC TESTING\n\n";

    struct TestCase {
        string name;
        int zone;
        int hour;
        int minute;
        long long expectedFare;
    };

    TestCase tests[] = {
        {"Inner off-peak", 1, 11, 0, 1000},
        {"Inner at 8:00", 1, 8, 0, 1200},
        {"Middle at 10:00", 2, 10, 0, 2000},
```

```
    {"Outer at 17:00", 3, 17, 0, 3600},
    {"Outer at 20:00", 3, 20, 0, 3000}
};

int totalTests = sizeof(tests) / sizeof(tests[0]);
int passed = 0;
long long totalFare = 0;

cout << "FARE TEST TABLE\n";

for (const TestCase& test : tests) {
    try {
        long long actual = calculateFare(
            test.zone, test.hour, test.minute
        );

        bool success = actual == test.expectedFare;

        cout << test.name << " | Expected: ";
        printMoney(test.expectedFare);
        cout << " | Actual: ";
        printMoney(actual);
        cout << " | " << (success ? "PASS" : "FAIL") << '\n';

        if (success) {
            passed++;
            totalFare += actual;
        }
```

```
    }
    catch (const exception& e) {
        cout << test.name << " | FAIL: "
             << e.what() << '\n';
    }
}

cout << "\nFIVE JOURNEY TOTAL\n";
cout << "Total fare: ";
printMoney(totalFare);
cout << '\n';

// Balance leaving exactly Rs. 50
long long balance = 6000;
long long fare = 1000;

cout << "\nMINIMUM BALANCE TEST\n";
bool allowed = canTravel(balance, fare);
cout << "Balance Rs. 60, fare Rs. 10: "
     << (allowed ? "PASS" : "FAIL") << '\n';

if (allowed) {
    balance -= fare;
}

cout << "Remaining balance: ";
printMoney(balance);
cout << '\n';
```

```
// Unknown zone test
cout << "\nINVALID ZONE TEST\n";
try {
   calculateFare(4, 12, 0);
   cout << "FAIL\n";
}
catch (const invalid_argument&) {
   cout << "PASS: Unknown zone rejected\n";
   passed++;
   totalTests++;
}

// Negative recharge test
cout << "\nNEGATIVE RECHARGE TEST\n";
long long rechargeBalance = 10000;
bool rechargeResult = recharge(rechargeBalance, -500);

cout << (!rechargeResult &&
       rechargeBalance == 10000
       ? "PASS: Negative recharge rejected\n"
       : "FAIL\n");

if (!rechargeResult && rechargeBalance == 10000) {
   passed++;
}
totalTests++;
```

```
    // Invalid hour input test
    cout << "\nINVALID HOUR INPUT TEST\n";
    int hour = 0;
    bool validHour = parseHour("abc", hour);

    cout << (!validHour
          ? "PASS: Letters rejected as hour\n"
          : "FAIL\n");

    if (!validHour) {
       passed++;
    }
    totalTests++;

    cout << "\nTEST SUMMARY\n";
    cout << "Tests passed: " << passed
         << " out of " << totalTests << '\n';

    return 0;
}
```